

Research Note: Low-Level Design (LLD) Practice Platform

Project: Takshaka — Low-Level Design Practice Platform

Deliverable: Research Note (2-Day Engineering Assignment, Section 7)

1. The Learner Problem

Preparing for Low-Level Design (LLD), object-oriented design, and machine-coding interviews presents a severe learning gap:

- **Easy to start, hard to evaluate:** A candidate can sketch classes for a Parking Lot or Elevator System, but has no objective mechanism to know if their class boundaries, abstractions, coupling, and trade-offs are actually sound without paying for expensive human mock interviews (\$150–\$300/hr).
 - **Misaligned automated judges:** Traditional competitive programming judges (LeetCode, HackerRank) only test *black-box algorithmic inputs and outputs*. They do not evaluate architectural design, single responsibility, encapsulation, or extensibility.
 - **Passive reading paralysis:** Candidates read GitHub repositories (e.g. *awesome-low-level-design*) and watch YouTube videos, but passive consumption fails to build real-time spatial and structural synthesis skills.
-

2. Existing Approaches Researched

We researched three primary benchmarks in the LLD and machine-coding ecosystem:

Benchmark 1: LLDCanvas (<https://www.lldcanvas.in/>)

- **Workflow:** Offers problem statements with staged hints, plain-text "Draft Notation" that generates UML diagrams, and an in-browser code editor.
- **Evaluation:** Self-directed. There is **no automated evaluation or scoring engine**. Candidates must consult community discussion threads to manually compare their solutions with others.
- **Takeaway:** High friction on validation; great at diagramming, but lacks automated, personalized feedback.

Benchmark 2: HelloInterview (<https://www.hellointerview.com/practice/low-level-design>)

- **Workflow:** Step-by-step guided practice (Requirements → Core Entities → Class Design → Implementation → Extensibility) led by an AI conversational bot.
- **Evaluation:** AI-assisted interview interaction.
- **Gaps:** Hard login walls and paywalls. Highly chat-heavy rather than visual; candidates write long conversational text responses instead of modeling architectural systems visually.

Benchmark 3: LLD Arena (<https://github.com/mightbeanshuu/lld-arena>)

- **Workflow:** Public dashboard with split-pane Monaco code editor (Java only).
 - **Evaluation:** Hybrid model. Uses local `javac` execution for unit tests, combined with an LLM prompt enforcing a strict JSON rubric scorecard.
 - **Takeaway:** Proved that **rubric-driven LLM evaluation with strict JSON schema output** produces actionable feedback. However, requiring code execution in an in-browser environment creates security, runtime sandboxing, and language-lock-in challenges.
-

3. Key Gaps Identified

Dimension	Existing Tools	Candidate Pain Point
Feedback Mechanism	Binary pass/fail unit tests OR unguided community forums	Ignores architectural principles (SOLID, coupling, cohesion)
Input Modality	100+ lines of boilerplate code OR unstructured text paragraphs	Candidates spend time debugging syntax instead of designing domain abstractions
Accessibility	Login walls, subscriptions, or local-only Docker setups	High barrier to quick, daily interview practice
Iteration Loop	One-off submission with ephemeral feedback	No immutable attempt history or side-by-side iterative comparison

4. Product Direction & MVP Synthesis

Based on these findings, we designed **Takshaka** with the following foundational decisions:

1. Visual Whiteboard Modeling over Raw Syntax:

- Candidates model their domain architecture using custom UML nodes (Classes, Interfaces, Abstract Classes, Enums) and typed relationships (Inheritance, Implementation, Composition, Association).
- This eliminates language syntax overhead while extracting clean, structured architectural signals (class responsibilities, methods, fields, and dependencies).

1. Rubric-Driven, Zero-Hallucination AI Evaluation:

- Rather than asking an LLM an open-ended question ("*Is this good?*"), the engine injects an **authoritative, multi-criterion rubric** tailored to the problem.
- The engine scores each criterion independently (1–5), providing concrete evidence cited directly from the candidate's design, alongside targeted concerns and actionable suggestions.

1. Immutable History & Iterative Retry Loop:

- Completed attempts are locked and immutable.
- Clicking "*Try Again*" clones the current graph into a fresh draft, enabling candidates to refactor their architecture and observe measurable score progression over multiple iterations.

1. Zero-Friction, Serverless Delivery:

- No login walls, no mandatory databases.
- Fast, client-side persistence with Next.js Turbopack and serverless evaluation hosted on Vercel.